

GNU Backgammon Mobile Tutor

Mission Statement and Product Philosophy

GNU Backgammon for Mobile

June 18, 2026

1 The Opportunity

GNU Backgammon is already one of the strongest analytical backgammon engines available. Its value is not in question. It can evaluate positions, rank moves, judge cube decisions, and expose mistakes with a precision few players can match.

The problem is not the engine.

The problem is the way that knowledge reaches the player.

The traditional desktop tutorial experience is powerful but awkward. It often presents judgement as numbers, tables, modal dialogs, and post-hoc analysis windows. This works for experts who already know what they are looking at. It works less well for players who are still learning how to see the board.

On mobile, copying that desktop interface would be a mistake. A phone is not a small desktop. A touch screen is not a mouse-driven analysis window. A mobile tutor should not be a compressed version of the PC program. It should be a different expression of the same intelligence.

The mission is therefore:

To turn GNU Backgammon from a powerful evaluator into a clear, visual, humane mobile coach.

Not by weakening the engine. Not by hiding the truth. Not by replacing GNUbg's judgement with vague encouragement. But by translating its judgement into a form that a player can understand directly on the board.

The mobile tutor should make the player feel:

“I see what I missed.”

That is the core product promise.

2 The Board Is Sacred

The board is the centre of backgammon. Every lesson ultimately lives there: the blot left exposed, the point not made, the anchor missed, the cube offered too early, the race misunderstood.

Desktop analysis often moves the player's attention away from the board and into secondary windows. That is exactly what mobile must avoid.

The mobile tutor should be board-first.

The player should not be pulled away into a spreadsheet of equities unless they ask for it. The first explanation should be visual, contextual, and immediate. A mistake should not primarily produce a dialog. It should produce a changed understanding of the position.

The tutor layer should live around and on top of the board:

- arrows showing the user's move and GNUbg's preferred move;
- subtle highlights for exposed blots;
- badges for hitting numbers;
- point-making highlights;
- a compact coach card that explains only the most important fact first.

The goal is not to display more information. The goal is to display the right information at the moment when it can still change the player's perception.

3 Mobile Is Not a 1:1 Port

A successful mobile GNUbg tutor must reject the idea of a literal desktop translation.

A desktop program can assume:

- large screens;
- mouse precision;
- multiple windows;
- dense tables;
- patient expert users;
- analysis as a separate mode after play.

A mobile tutor must assume:

- small screens;
- touch interaction;

- short attention windows;
- board visibility as the main constraint;
- players who may be learning during live play;
- explanation as part of the move loop.

The mobile version should not ask:

“How do we fit the desktop tutor onto a phone?”

It should ask:

“What would GNUbg teach if it could point directly at the board?”

This is the novelty. The product is not merely GNUbg on Android. The product is GNUbg reinterpreted for a tactile, visual, coach-like medium.

4 From Judgement to Coaching

GNUbg can say which move is best. That is judgement.

A tutor must help the player understand why. That is coaching.

The mobile tutor should bridge this gap carefully.

The engine remains the authority on evaluation. It decides:

- best move;
- move ranking;
- equity loss;
- cube correctness;
- severity of error.

The mobile layer should not pretend to be a second backgammon brain. It should not invent reasons that the engine did not prove. Instead, it should compute measurable differences between the user’s move and GNUbg’s preferred move.

Examples of grounded facts include:

- the user’s move leaves 17 hitting numbers while the best move leaves 6;
- GNUbg’s move makes the 5-point while the user’s move does not;
- the user exposes an extra blot;

- the user gives up race equity;
- the cube decision crosses the take/pass threshold;
- gammon danger changes the cube decision.

Only after such facts are computed should the app generate language.

This gives the tutor its governing rule:

Every explanation must be grounded in measurable board facts and GNUbg's evaluation.

If the app cannot identify a reliable reason, it should say less:

GNUbg prefers another move by 0.062. Show best move?

That is better than confidently teaching a false lesson.

5 The Tutor Should Be Kind, Not Soft

The mobile tutor should be honest without being punitive.

Backgammon is already emotionally sharp. A player makes a move, gets hit, loses a gammon, drops a cube, or watches a match turn. A tutor that scolds will quickly become unwelcome.

The tone should be calm, precise, and useful.

Instead of:

Blunder. Your move is bad.

Say:

Mistake: -0.084.

Main reason: your move leaves many more shots.

Instead of:

Wrong. Best move is 13/8 6/5.

Say:

GNUbg prefers 13/8 6/5.

It makes the 5-point and leaves fewer direct shots.

The goal is not to protect the user from mistakes. The goal is to make mistakes productive.

6 The Try Again Loop

The strongest learning interaction is not passive explanation. It is retrieval and correction.

When the user makes a significant mistake, the app should offer:

- Show best move;
- Try again;
- More detail.

The *Try again* path is essential.

It restores the pre-move position and lets the player search again, now with a hint or without one depending on settings. If the player finds the better move, the tutor confirms the idea.

This changes the emotional structure of tutorial mode.

The desktop pattern is:

You moved. Here is why you were wrong.

The mobile coach pattern is:

You missed something. Look again.

That is a fundamentally better learning loop.

7 Progressive Disclosure

The tutor should not show all of GNUbg at once.

Backgammon analysis has many layers: move equity, cubeless equity, cubeful equity, win percentage, gammons, backgammons, rollout confidence, cube efficiency, match score effects. These are valuable, but they are not always the first thing the player needs.

The mobile tutor should reveal information progressively.

First level:

Mistake: -0.084.
Better: 13/8 6/5.

Second level:

Main reason: safety.
Your move leaves 17 shots; GNUbg's move leaves 6.

Third level:

Your move equity: +0.112.
Best move equity: +0.196.
Loss: -0.084.

Advanced level:

Win %, gammon %, backgammon %, cubeful/cubeless data, rollout options.

This makes the interface useful for beginners without insulting experts. It also allows a “Classic” mode without forcing classic density onto every player.

8 One Renderer, Multiple Depths

The tutor should not become three separate products.

“Gentle Coach”, “Serious Coach”, and “Classic GNUbg” should be understood as different disclosure levels of the same system.

The underlying data stays the same:

- GNUbg evaluation;
- feature deltas;
- tutor hint;
- board annotations;
- optional numeric detail.

The setting only changes:

- how often interruptions occur;
- how much detail opens by default;
- whether explanations use plain language or technical language;
- whether full move tables and equities are shown immediately.

This avoids creating parallel code paths. It keeps the product coherent.

9 Visual Explanation Is the Mobile Advantage

Desktop GNUbg is strong because it calculates.

Mobile GNUbg can shine because it can point.

Shot counting is a perfect example. A desktop tutor may tell the user that a move leaves too many shots. A mobile tutor can show the exposed checker, mark the opponent's attacking checkers, and display small dice badges for the rolls that hit.

That is not cosmetic. It changes understanding.

The same principle applies to:

- best-move arrows;
- user-move comparison;
- made-point highlights;
- blots at risk;
- cube take/pass meters;
- race bars;
- before/after board toggles.

A good mobile tutor should often require less text than the desktop tutor, because the board itself becomes the explanation.

10 The Cube Tutor as a Separate Strength

Cube decisions are among the hardest parts of backgammon and among the weakest areas of most mobile backgammon products.

GNUbg already has the analytical strength to judge cube decisions. The mobile UI can make those judgements understandable.

A cube tutor should not merely say:

Double / Take / Pass.

It should show:

- current winning chances;
- take point;
- gammon danger;
- match score context;

- whether the position is no double, double/take, double/pass, or too good.

The cube should become visual.

A player should be able to see why a take is barely correct, why a pass is forced, or why waiting loses the market. This is one of the clearest opportunities to make the mobile version feel more modern and more teachable than the desktop experience.

11 Natural Language Must Be Grounded

The dream feature is human-sounding explanation:

Make your 5-point. It strengthens your board and makes your opponent's hits less dangerous.

But this is also the danger zone.

GNUbg does not naturally output “reasons” in human language. It outputs evaluations. Any prose layer is therefore an interpretation. If that interpretation is wrong, the app becomes worse than silent: it teaches false lessons with confidence.

The safe approach is feature-delta grounding.

The app should first compute facts:

- Did GNUbg's move make a key point?
- Did the user's move fail to make it?
- Did shot count change?
- Did blot count change?
- Did race equity change?
- Did cube take/pass math cross a threshold?

Only then should it choose a template.

Natural language should be generated from facts, not from vibes.

This means the first version should have a small vocabulary of reliable concepts:

- Safety;
- Shot count;
- Point-making;
- Exposure;
- Race;

- Cube take/pass;
- Gammon danger.

More subtle concepts such as timing, containment, passivity, or duplication should only be added when the app can compute them reliably.

The tutor earns trust by being conservative.

12 The Product Should Teach Patterns Over Time

A single tutor event helps with one move.

A great tutor helps the player notice recurring leaks.

Over time, the app should be able to say:

Your most common mistake this session: leaving shots while ahead in the race.

Or:

You missed three chances to make key inner-board points.

This turns GNUbg from a position evaluator into a training companion.

The long-term vision is not just game review. It is pattern review.

The app should eventually help answer:

- What kind of mistakes do I make?
- Do I double too late?
- Do I drop too much?
- Do I leave too many shots?
- Do I miss point-making opportunities?
- Am I too passive in contact positions?

This is where retention comes from. Players return because the app knows what they are trying to improve.

13 Rollouts Must Be Treated Carefully

Rollouts are part of GNUbg's depth, but mobile devices are constrained.

A real rollout can be slow, battery-intensive, and confusing if presented casually. The tutor should not encourage users to tap a heavy compute button without understanding what will happen.

Rollout features should therefore be:

- clearly labelled as deeper analysis;
- bounded by trial limits;
- cancelable;
- optionally deferred;
- visually distinct from instant evaluation.

The default tutor should rely on fast evaluation first. Rollouts belong in advanced detail, not in the primary correction loop.

14 What the First Version Should Prove

The first mobile tutor should not try to implement every idea.

It should prove one central experience:

I made a move, GNUbg judged it, and the app showed me visually and kindly what I missed.

The first implementation should focus on:

1. detecting a significant move error;
2. showing a compact coach card;
3. displaying the best move;
4. offering *Try again*;
5. drawing board arrows;
6. showing shot-count differences when computable.

That alone would already be a major advance over most mobile backgammon tutors.

The first version should avoid:

- full desktop-style move tables as the primary UI;
- excessive modal dialogs;
- ungrounded natural-language explanations;
- complex rollout workflows;
- broad concept classification before the feature-delta layer is reliable.

The MVP must be small enough to build and strong enough to define the product.

15 The Design Standard

Every tutorial feature should pass four tests.

Is it grounded?

Can we trace the explanation back to GNUbg evaluation or deterministic board features?

Is it visible?

Can the user see the point on the board, not just read it?

Is it interrupt-light?

Does it preserve flow and avoid unnecessary modal friction?

Is it teachable?

Does it help the user recognise the same idea next time?

If a feature fails these tests, it belongs later or not at all.

16 The Mission

The mission of GNU Backgammon Mobile Tutor is not to reproduce the desktop tutor.

It is to take the strongest backgammon engine available and give it a mobile-native teaching interface.

The desktop program calculates brilliantly. The mobile app must communicate brilliantly.

It should not merely report mistakes. It should help players see them.

It should not bury the board under analysis. It should make the board speak.

It should not overwhelm beginners with expert tables. It should let depth unfold as the player asks for it.

It should not invent explanations. It should ground every lesson in facts.

If this succeeds, the result is not just GNUbg on a phone. It is a new kind of backgammon tutor: precise enough for experts, clear enough for learners, and tactile enough that the lesson happens directly under the player's fingers.